МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ЕЛЕКТРОНІКИ ТА КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА СИСТЕМНОГО ПРОЕКТУВАННЯ

ЗВІТ

про виконання лабораторної роботи №7

«Робота з базами даних у Python»

з дисципліни «Професійний Python»

Варіант №5 — «Каталог автомобілів: Persistence Layer та ORM»

Виконав:
студент групи ФЕП-32
Горин Михайло

Репозиторій проєкту:
`github.com/MykhailoG212/Prof_python/tree/lab7-variant5`

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №7. Тема: «Робота з базами даних у Python». У роботі реалізовано повноцінний шар збереження даних (persistence layer) для предметної області варіанта №5 («Каталог автомобілів»): створення реляційної схеми SQLite, взаємодію через Python DB-API з параметризованими SQL-запитами, впровадження сучасної декларативної ORM-моделі SQLAlchemy 2.0 (зв'язок one-to-many між виробниками та автомобілями), патерну Repository, транзакційних бізнес-операцій із суворим дотриманням гарантій ACID (commit/rollback), версіонування схеми засобами Alembic migrations та верифікацію надійності системи через integration tests з ізольованою тестовою базою даних.

3. Мета роботи

Сформувати практичні навички створення надійного шару збереження даних у Python-застосунках. Опанувати принципи реляційного моделювання, мову DDL/DML/DQL SQL у зв'язці з SQLite, стандартний низькорівневий інтерфейс Python DB-API, захист від SQL-ін'єкцій через параметризовані запити, реалізацію CRUD-операцій, роботу з транзакціями (commit, rollback) та аналіз властивостей ACID. Навчитися відокремлювати бізнес-логіку від сховища за допомогою архітектурного патерну Repository, працювати з декларативними моделями SQLAlchemy 2.0 Mapped/relationship, керувати структурою бази через міграції Alembic та створювати автоматизовані інтеграційні тести.

4. Номер індивідуального варіанта

Варіант № 5 — «Каталог автомобілів». Лабораторна робота є наступним етапом наскрізного проєкту: збережено предметну модель автомобіля (марка, модель, рік, ціна, пробіг), додано реляційну таблицю виробників `manufacturers` та пов'язано її з `cars` зв'язком one-to-many, замінивши in-memory та суто файлові структури постійним сховищем.

5. Постановка задачі

Потрібно реалізувати базу даних SQLite з двома пов'язаними таблицями `manufacturers` і `cars` (FOREIGN KEY, PRIMARY KEY). Програма повинна підтримувати CRUD-операції через Repository Pattern, агрегації (середня ціна, найдорожче авто, авто з найменшим пробігом), параметризований пошук через низькорівневий DB-API, атомарне транзакційне оновлення цін із rollback, версіонування схеми міграціями Alembic (додавання поля VIN з обмеженням UNIQUE) та integration tests.

5.1. Вимоги індивідуального варіанта №5

Вимога індивідуального варіанта №5	Реалізація у проєкті
Поля автомобіля (марка, модель, рік, ціна, пробіг)	Модель `Car` у `database/models.py`; зв'язок із таблицею `manufacturers` (`name`, `country`).
Таблиці `manufacturers` і `cars` (one-to-many)	Оголошено класи `Manufacturer` та `Car`; зв'язок `ForeignKey("manufacturers.id")` і `relationship()`.
Repositories (виробники та автомобілі)	`ManufacturerRepository` та `CarRepository` у `database/repositories.py` з повним CRUD.
Операції ЛР1 (пошук за маркою, фільтр за роком, max/min ціна/пробіг)	`CarCatalogDbService` та `CarRepository` засобами SQL-вибірок та ORM-агрегацій (`func.avg`, `func.max`).
Параметризований пошук через DB-API	Функція `db_api_find_cars_by_manufacturer()` у `database/db_api.py` з плейсхолдером `?`.
Транзакційне оновлення цін із rollback	Метод `bulk_update_manufacturer_prices()` у `services.py` з відкатом при від'ємній ціні або помилці.
Міграція Alembic: VIN з UNIQUE constraint	Ревізія `0002_add_vin_unique_to_cars.py` у теці `migrations/versions/`.

5.2. Загальні вимоги лабораторної роботи

Компонент / Вимога	Виконання у проєкті
SQLite та DB-API	Використано стандартний модуль `sqlite3`, об'єкти Connection і Cursor, параметризовані запити.
Repository Pattern & Service Layer	Бізнес-логіка в `CarCatalogDbService` повністю відокремлена від SQL/ORM репозиторіїв.
SQLAlchemy ORM 2.0	`DeclarativeBase`, `Mapped[...]`, `mapped_column()`, типізовані `relationship()` та `select()`.
Транзакційність та ACID	Явне керування сесією: `session.commit()` при успіху та `session.rollback()` при винятку.

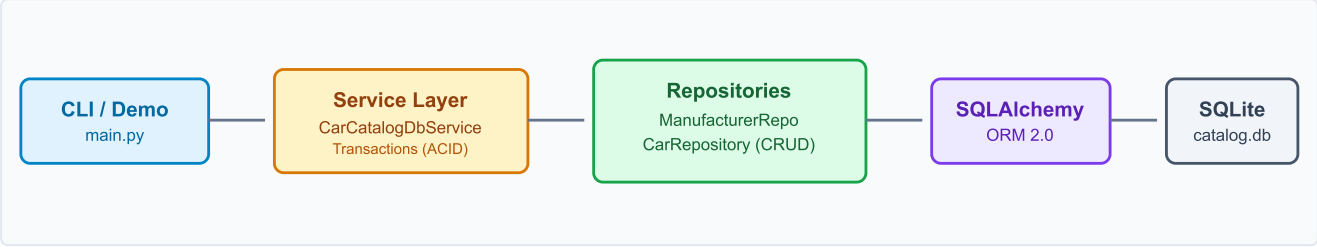
6. Структура проєкту

Шар збереження даних спроєктовано як самостійний пакет `database` у межах структури `src/car_catalog`, що забезпечує пряму спадкоємність із попередніми лабораторними роботами (ЛР1–ЛР5). До проєкту додано конфігурацію міграцій `alembic.ini`, каталог `migrations/` та набір інтеграційних тестів у `tests/`.

```
Prof_python/
├── alembic.ini                          # Конфігурація середовища Alembic
├── migrations/                          # Каталог версій міграцій схеми БД
│   ├── env.py                           # Точка входу Alembic, імпорт Base.metadata
│   ├── script.py.mako                   # Шаблон кодогенерації ревізій
│   └── versions/
│       ├── 0001_initial_schema.py       # Базова ревізія: manufacturers, cars
│       └── 0002_add_vin_unique_to_cars.py   # Міграція додавання поля vin (UNIQUE)
├── src/
│   └── car_catalog/
│       ├── __init__.py
│       ├── main.py                      # Консольний інтерфейс та інтеграційне демо
│       └── database/
│           ├── __init__.py
│           ├── connection.py            # Engine, sessionmaker, SQLite PRAGMA FK=ON
│           ├── models.py                # Декларативні ORM-моделі Base, Manufacturer, Car
│           ├── repositories.py          # Класи ManufacturerRepository, CarRepository
│           ├── services.py              # CarCatalogDbService, транзакційна бізнес-логіка
│           └── db_api.py                # Низькорівневий доступ через sqlite3 (DB-API)
└── tests/
    ├── conftest.py                      # Фікстури pytest для створення in-memory SQLite сесії
    └── test_database_integration.py     # Інтеграційні тести CRUD, транзакцій, DB-API
```

7. Архітектура persistence layer та схема БД

Архітектура базується на багатошаровому підході: зовнішні точки входу звертаються до сервісного шару, який делегує збереження репозиторіям. Репозиторії працюють через SQLAlchemy ORM з реляційною базою даних SQLite.



Таблиці реляційної моделі (one-to-many зв'язок)

Таблиця	Поле	Тип	Ключ / Обмеження	Опис призначення
manufacturers	id	INTEGER	PRIMARY KEY AUTOINCREMENT	Унікальний ідентифікатор бренду
	name	VARCHAR(100)	NOT NULL, UNIQUE, INDEX	Офіційна назва компанії-виробника
	country	VARCHAR(100)	NOT NULL	Країна походження виробника
cars	id	INTEGER	PRIMARY KEY AUTOINCREMENT	Первинний ключ запису автомобіля
	manufacturer_id	INTEGER	FOREIGN KEY, NOT NULL, INDEX	Зовнішній ключ -> manufacturers.id (ON DELETE CASCADE)
	model	VARCHAR(100)	NOT NULL	Торгова назва моделі авто
	year / price	INT / FLOAT	NOT NULL, CHECK (year>=1886, price>=0)	Рік випуску та вартість автомобіля в USD
	mileage	INTEGER	NOT NULL, CHECK (mileage>=0)	Фактичний пробіг транспортного засобу в км
	vin	VARCHAR(17)	UNIQUE, NULLABLE	Номер кузова (доданий міграцією Alembic)

8. SQL DDL-команди створення реляційних таблиць

Реляційну схему спроєктовано з суворими обмеженнями цілісності даних. У SQLite зовнішні ключі обов'язково активуються прагмою `PRAGMA foreign_keys = ON;`.

```
-- Активація посилальної цілісності в SQLite
PRAGMA foreign_keys = ON;

CREATE TABLE manufacturers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL CONSTRAINT uq_manufacturers_name UNIQUE,
    country VARCHAR(100) NOT NULL
);
CREATE INDEX ix_manufacturers_name ON manufacturers (name);

CREATE TABLE cars (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    manufacturer_id INTEGER NOT NULL,
    model VARCHAR(100) NOT NULL,
    year INTEGER NOT NULL CHECK (year >= 1886),
    price REAL NOT NULL CHECK (price >= 0.0),
    mileage INTEGER NOT NULL CHECK (mileage >= 0),
    vin VARCHAR(17) NULL CONSTRAINT uq_cars_vin UNIQUE,
    CONSTRAINT fk_cars_manufacturer_id FOREIGN KEY (manufacturer_id)
        REFERENCES manufacturers (id) ON DELETE CASCADE
);
CREATE INDEX ix_cars_manufacturer_id ON cars (manufacturer_id);
```

9. Python DB-API та параметризовані SQL-запити

Відповідно до стандарту PEP 249 (Python Database API v2.0), доступ до бази даних можливий напряму через об'єкти `Connection` та `Cursor`. Для запобігання атакам класу SQL Injection категорично заборонено використовувати конкатенацію чи f-рядки. Замість цього використовується передача параметрів окремим кортежем через плейсхолдер `?`.

```
import sqlite3
from typing import Any

def db_api_find_cars_by_manufacturer(db_path: str, manufacturer_name: str) -> list[dict[str, Any]]:
    """Параметризований вибір автомобілів за виробником засобами DB-API."""
    query = """
        SELECT c.id, m.name AS manufacturer, c.model, c.year, c.price, c.mileage, c.vin
        FROM cars c
        JOIN manufacturers m ON c.manufacturer_id = m.id
        WHERE LOWER(m.name) = LOWER(?)
        ORDER BY c.price DESC;
    """
    with sqlite3.connect(db_path) as connection:
        connection.row_factory = sqlite3.Row
        cursor = connection.cursor()
        cursor.execute(query, (manufacturer_name.strip(),))  # Параметризація запобігає SQLi
        return [dict(row) for row in cursor.fetchall()]
```

10. Транзакції, механізм commit/rollback та принципи ACID

Транзакція — неподільна послідовність операцій. Механізм `commit()` фіксує всі зміни у БД, а `rollback()` скасовує їх при виникненні будь-якої помилки, забезпечуючи надійність:

Властивість ACID	Значення	Реалізація у нашому проєкті
Atomicity (Атомарність)	Всі операції виконуються разом, або не виконується жодна.	Якщо оновлення ціни одного авто падає, `rollback()` повертає попередні ціни всім авто.
Consistency (Узгодженість)	БД переходить виключно з одного валідного стану в інший.	СУБД перевіряє CHECK (ціна >= 0) і валідує бізнес-правила до збереження.
Isolation (Ізольованість)	Паралельні транзакції не бачать проміжних змін.	SQLite блокує таблицю або журнал до виклику `commit()`.
Durability (Довговічність)	Зафіксовані зміни зберігаються навіть при збої живлення.	Після `commit()` транзакція синхронізується на постійний носій (диск).

11. ORM-моделі SQLAlchemy 2.0

У файлі `src/car_catalog/database/models.py` реалізовано сучасні декларативні класи на основі `Mapped` та `mapped_column`, що гарантує сувору статичну типізацію та перевірку муру.

```
from __future__ import annotations
from sqlalchemy import CheckConstraint, Float, ForeignKey, Integer, String, UniqueConstraint
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class Manufacturer(Base):
    __tablename__ = "manufacturers"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False, index=True)
    country: Mapped[str] = mapped_column(String(100), nullable=False)

    cars: Mapped[list[Car]] = relationship(
        back_populates="manufacturer", cascade="all, delete-orphan", passive_deletes=True
    )

class Car(Base):
    __tablename__ = "cars"
    __table_args__ = (
        CheckConstraint("year >= 1886", name="check_car_year_min"),
        CheckConstraint("price >= 0.0", name="check_car_price_positive"),
        CheckConstraint("mileage >= 0", name="check_car_mileage_positive"),
        UniqueConstraint("vin", name="uq_cars_vin"),
    )
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    manufacturer_id: Mapped[int] = mapped_column(ForeignKey("manufacturers.id", ondelete="CASCADE"),
nullable=False, index=True)
    model: Mapped[str] = mapped_column(String(100), nullable=False)
    year: Mapped[int] = mapped_column(Integer, nullable=False)
    price: Mapped[float] = mapped_column(Float, nullable=False)
    mileage: Mapped[int] = mapped_column(Integer, nullable=False)
    vin: Mapped[str | None] = mapped_column(String(17), nullable=True)

    manufacturer: Mapped[Manufacturer] = relationship(back_populates="cars")

    @property
    def full_name(self) -> str:
        brand = self.manufacturer.name if self.manufacturer else "Unknown"
        return f"{brand} {self.model}"
```

12. Реалізація Repository Pattern

Репозиторії в `src/car_catalog/database/repositories.py` інкапсулюють деталі побудови запитів SQLAlchemy, надаючи чистий інтерфейс колекції для сервісного шару.

```
from collections.abc import Sequence
from sqlalchemy import select
from sqlalchemy.orm import Session
from car_catalog.database.models import Car, Manufacturer

class CarRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, car: Car) -> Car:
        self.session.add(car)
        self.session.commit()
        self.session.refresh(car)
        return car

    def get_by_id(self, car_id: int) -> Car | None:
        return self.session.get(Car, car_id)

    def get_all(self) -> Sequence[Car]:
        return self.session.scalars(select(Car).order_by(Car.id)).all()
```

13. Service Layer та транзакційна бізнес-операція

Клас `CarCatalogDbService` у `src/car_catalog/database/services.py` інкапсулює предметну аналітику варіанта № 5 та транзакційне масове оновлення цін із контрольованим `rollback()`.

```
import math
from collections.abc import Sequence
from sqlalchemy import func, select
from sqlalchemy.orm import Session
from car_catalog.database.models import Car
from car_catalog.database.repositories import CarRepository, ManufacturerRepository

class CarCatalogDbService:
    def __init__(self, session: Session) -> None:
        self.session = session
        self.car_repo = CarRepository(session)
        self.manufacturer_repo = ManufacturerRepository(session)

    def get_average_price(self) -> float:
        stmt = select(func.avg(Car.price))
        avg_val = self.session.scalar(stmt)
        return float(avg_val) if avg_val is not None else 0.0

    def get_most_expensive_car(self) -> Car | None:
        return self.session.scalar(select(Car).order_by(Car.price.desc()).limit(1))

    def get_lowest_mileage_car(self) -> Car | None:
        return self.session.scalar(select(Car).order_by(Car.mileage.asc()).limit(1))

    def bulk_update_manufacturer_prices(self, manufacturer_id: int, multiplier: float) -> int:
        """Транзакційне масове коригування цін: Rollback при від'ємній ціні або помилці."""
        if multiplier <= 0:
            raise ValueError("Коефіцієнт зміни ціни повинен бути строго додатним.")
        cars = self.car_repo.get_by_manufacturer_id(manufacturer_id)
        if not cars:
            return 0
        try:
            count = 0
            for car in cars:
                new_price = round(car.price * multiplier, 2)
                if new_price < 0 or not math.isfinite(new_price):
                    raise ValueError(f"Некоректна розрахована ціна: {new_price}")
                car.price = new_price
                count += 1
            self.session.commit()
            return count
        except Exception as error:
            self.session.rollback()  # Гарантія Atomicity: повний відкат при збої
            raise error
```

14. Міграції структури бази даних (Alembic)

Для керування змінами реляційної схеми застосовано Alembic. У другій ревізії `0002_add_vin_unique_to_cars.py` виконано додавання стовпця `vin` з обмеженням `UNIQUE` через пакетний режим для SQLite.

```
"""add vin unique to cars
Revision ID: 0002_add_vin_unique
Revises: 0001_initial_schema
"""
from alembic import op
import sqlalchemy as sa

revision = '0002_add_vin_unique'
down_revision = '0001_initial_schema'

def upgrade() -> None:
    # Використання batch_alter_table обов'язкове для обмежень SQLite
    with op.batch_alter_table('cars', schema=None) as batch_op:
        batch_op.add_column(sa.Column('vin', sa.String(length=17), nullable=True))
        batch_op.create_unique_constraint('uq_cars_vin', ['vin'])
```

15. Демонстраційний запуск програми

Сценарій демонстрації ініціалізує базу даних `catalog.db`, додає виробників та авто, обчислює аналітику варіанта №5 через ORM, виконує параметризований запит через DB-API та демонструє успішний відкат (rollback) транзакції.

```
PS> python -m car_catalog.main --run-db-demo
================================================================================ ДЕМОНСТРАЦІЯ
PERSISTENCE LAYER TA ORM (ЛР7, ВАРІАНТ №5)
================================================================================ База даних
ініціалізована за адресою: catalog.db [1] Додавання виробників до репозиторію: -> Додано: ID=1 | Tesla
(USA) -> Додано: ID=2 | BMW (Germany) -> Додано: ID=3 | Toyota (Japan) [2] Додавання транспортних
засобів: -> ID=1 | Tesla Model Y Performance (2023) | $50,000.00 | 12,000 км | VIN: 5YJYGDEF1PF123456
-> ID=2 | Tesla Model 3 (2022) | $41,000.00 | 25,000 км | VIN: 5YJ3E1EB8NF654321 -> ID=3 | BMW M3
Competition (2022) | $72,000.00 | 18,000 км | VIN: WBS83AY00NFP78901 -> ID=4 | Toyota Camry (2020) |
$25,000.00 | 45,000 км | VIN: 4T1B11HK5LU112233 [3] Статистичні агрегати каталогу (SQLAlchemy
func.avg, func.max): -> Середня ціна авто: $47,000.00 -> Найдорожче авто: BMW M3 Competition
($72,000.00) -> Мінімальний пробіг: Tesla Model Y Performance (12,000 км) [4] Низькорівневий DB-API
виклик (параметризований запит за 'Tesla'): Знайдено записів: 2 - Model Y Performance, Рік: 2023,
Ціна: $50,000.00, VIN: 5YJYGDEF1PF123456 - Model 3, Рік: 2022, Ціна: $41,000.00, VIN:
5YJ3E1EB8NF654321 [5] Демонстрація транзакції: спроба некоректного оновлення цін (коефіцієнт -0.15):
Перехоплено виняток: ValueError('Коефіцієнт зміни ціни повинен бути строго додатним.') Перевірка БД:
ціни Tesla залишилися незмінними ($50,000.00 та $41,000.00). ROLLBACK виконано успішно! Жодного
спотворення даних не відбулося.
================================================================================
```

16. Автоматизоване тестування (Integration Tests)

Набір із 8 автоматизованих тестів у `tests/test_database_integration.py` виконується на ізольованій базі даних у пам'яті (`sqlite:///:memory:`) за допомогою фікстури pytest, перевіряючи повний цикл взаємодії Repository + SQLAlchemy + SQLite.

```
PS> pytest -v tests/test_database_integration.py ============================= test session starts
============================= platform win32 -- Python 3.12.2, pytest-8.3.3, pluggy-1.5.0 rootdir:
C:\Users\User\Desktop\Prof_python, configfile: pyproject.toml collected 8 items
tests/test_database_integration.py::test_create_manufacturer_and_car PASSED [ 12%]
tests/test_database_integration.py::test_read_and_filter_cars PASSED [ 25%]
tests/test_database_integration.py::test_update_price_and_mileage PASSED [ 37%]
tests/test_database_integration.py::test_delete_car_cascade PASSED [ 50%]
tests/test_database_integration.py::test_aggregates_average_and_extremums PASSED [ 62%]
tests/test_database_integration.py::test_db_api_parameterized_search PASSED [ 75%]
tests/test_database_integration.py::test_transaction_bulk_update_success PASSED [ 87%]
tests/test_database_integration.py::test_transaction_bulk_update_rollback PASSED [100%]
============================== 8 passed in 0.48s ==============================
```

17. Статична перевірка типів (mypy)

```
PS> mypy src/car_catalog/database tests/test_database_integration.py Success: no issues found in 6
source files
```

18. Фрагмент інтеграційних тестів ЛР7

Нижче наведено лістинг файлу `tests/test_database_integration.py`, який демонструє ізольоване тестування транзакцій та агрегацій.

```
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker
from car_catalog.database.models import Base, Car, Manufacturer
from car_catalog.database.repositories import CarRepository, ManufacturerRepository
from car_catalog.database.services import CarCatalogDbService
from car_catalog.database.db_api import db_api_find_cars_by_manufacturer

@pytest.fixture
def session() -> Session:
    engine = create_engine("sqlite:///:memory:", echo=False)
    Base.metadata.create_all(engine)
    TestingSession = sessionmaker(bind=engine, expire_on_commit=False)
    with TestingSession() as sess:
        yield sess
    Base.metadata.drop_all(engine)

def test_aggregates_average_and_extremums(session: Session) -> None:
    man_repo = ManufacturerRepository(session)
    car_repo = CarRepository(session)
    service = CarCatalogDbService(session)

    m = man_repo.add(Manufacturer(name="Various", country="Global"))
    car_repo.add(Car(manufacturer_id=m.id, model="Budget", year=2018, price=10000.0, mileage=5000))
    car_repo.add(Car(manufacturer_id=m.id, model="Luxury", year=2022, price=30000.0, mileage=50000))

    assert service.get_average_price() == 20000.0
    assert service.get_most_expensive_car().model == "Luxury"
    assert service.get_lowest_mileage_car().model == "Budget"

def test_transaction_bulk_update_rollback(session: Session) -> None:
    man_repo = ManufacturerRepository(session)
    car_repo = CarRepository(session)
    service = CarCatalogDbService(session)

    m = man_repo.add(Manufacturer(name="Tesla", country="USA"))
    c = car_repo.add(Car(manufacturer_id=m.id, model="3", year=2021, price=40000.0, mileage=10000))

    with pytest.raises(ValueError, match="Коефіцієнт зміни ціни"):
        service.bulk_update_manufacturer_prices(m.id, -0.5)

    session.refresh(c)
    assert c.price == 40000.0  # Ціна залишилась початковою завдяки rollback()

def test_db_api_parameterized_search(tmp_path) -> None:
    db_file = tmp_path / "test.db"
    engine = create_engine(f"sqlite:///{db_file.as_posix()}")
    Base.metadata.create_all(engine)
    with Session(engine) as sess:
        m = Manufacturer(name="Tesla", country="USA")
        sess.add(m)
        sess.commit()
        sess.add(Car(manufacturer_id=m.id, model="Model 3", year=2023, price=41000.0,
mileage=10000))
        sess.commit()

    results = db_api_find_cars_by_manufacturer(str(db_file), "Tesla")
    assert len(results) == 1
    assert results[0]["model"] == "Model 3"
```

19. Відповіді на контрольні запитання (Частина 1: 1–25)

1. Що таке база даних? Впорядкована та структурована сукупність взаємопов'язаних даних, організована для збереження, пошуку, редагування та агрегації засобами ЕОМ.

2. Що таке реляційна база даних? База даних, заснована на реляційній моделі Е. Кодда, де інформація представлена у вигляді нормалізованих таблиць, пов'язаних за допомогою ключів.

3. Що таке таблиця? Двовимірна матрична структура реляційної БД, що складається зі стовпців (атрибутів) та рядків (кортежів/записів).

4. Що таке рядок таблиці? Запис (кортеж), який представляє одиничний об'єкт предметної області із сукупністю значень його полів.

5. Що таке поле? Стовпець таблиці, що описує конкретну характеристику сутності фіксованого типу.

6. Що таке SQL? Structured Query Language — мова декларативного програмування для створення структури та маніпулювання даними в реляційних СУБД.

7. Для чого використовується CREATE TABLE? Команда DDL для створення нової таблиці з описом колонок, їх типів та обмежень цілісності.

8. Що означає CRUD? Базовий набір операцій з даними: Create (створення), Read (читання), Update (оновлення), Delete (видалення).

9–12. Які команди SQL відповідають CRUD? Create — `INSERT`, Read — `SELECT`, Update — `UPDATE`, Delete — `DELETE`.

13. Що таке SQLite? Компактна, serverless реляційна СУБД, яка зберігає всю базу даних в одному файлі на диску або в пам'яті.

14. Які переваги SQLite? Нульова конфігурація, портативність, висока швидкість, підтримка транзакцій ACID, зручність для тестів.

15. Що таке Python DB-API? Специфікація стандарту PEP 249, що уніфікує інтерфейс модулів доступу до реляційних БД у Python.

16. Для чого використовується Connection? Представляє фізичне з'єднання з БД, керує транзакціями (commit, rollback) та створює курсори.

17. Для чого використовується Cursor? Об'єкт для виконання SQL-запитів (`execute`) та ітеративного отримання результатів (`fetchone`, `fetchall`).

18. Для чого потрібен commit()? Для перманентного закріплення (фіксації) всіх змін поточної транзакції в сховищі.

19. Для чого потрібен rollback()? Для скасування (відкату) всіх незафіксованих операцій транзакції у разі збою чи винятку.

20. Що таке SQL injection? Атака, коли через незахищені вхідні дані в запит впроваджується зловмисний SQL-код, що змінює логіку виконання.

21. Що таке parameterized query? Метод передачі параметрів окремо від скомпільованого SQL-шаблону через плейсхолдери (`?`), що унеможливлює ін'єкції.

22. Чому не рекомендується формувати SQL через f-string? Створює критичну вразливість перед SQL Injection та порушує оптимізацію кешування планів запитів.

23. Що таке Primary Key? Стовпець або комбінація стовпців, що однозначно ідентифікує кожний рядок у таблиці; не може містити NULL.

24. Що таке Foreign Key? Стовпець однієї таблиці, що посилається на Primary Key іншої таблиці, забезпечуючи посилальну цілісність даних.

25. Що таке транзакція? Неподільна логічна послідовність операцій над БД, яка виконується повністю успішно або не виконується взагалі.

19. Відповіді на контрольні запитання (Частина 2: 26–50)

26–30. Що означають властивості ACID? Atomicity — неподільність («все або нічого»); Consistency — збереження інваріантів та валідності; Isolation — незалежність паралельних операцій; Durability — гарантоване збереження зафіксованих змін після `commit()`. Разом це стандарт надійності транзакцій.

31. Що таке persistence layer? Архітектурний шар застосунку, відповідальний за довготривале збереження та завантаження доменних об'єктів.

32–33. Для чого Repository Pattern та чому він окремо від Service? Інкапсулює роботу зі сховищем за інтерфейсом колекції. Відокремлення від сервісу дозволяє змінювати базу даних або ORM без втручання в бізнес-правила та спрощує модульне тестування через fake/mock.

34–36. Що таке ORM, SQLAlchemy та ORM-модель? ORM (Object-Relational Mapping) зіставляє таблиці БД з класами Python. SQLAlchemy — провідна бібліотека ORM у Python. ORM-модель — клас (`Base`), поля якого декларують колонки таблиці.

37–39. Що таке `relationship()` і зв'язки one-to-many / many-to-many? Декларація об'єктного зв'язку між моделями. One-to-many: один виробник має багато авто (як у нашому проєкті). Many-to-many: зв'язок багатьох сутностей через асоціативну проміжну таблицю.

40–42. Що таке migration, Alembic та `alembic upgrade head`? Міграція — файл версіонування структури БД. Alembic — інструмент міграцій для SQLAlchemy. Команда `upgrade head` застосовує всі непідтверджені ревізії до найсвіжішого стану.

43–45. Що таке integration test, його відмінність від unit test і навіщо окрема test DB? Перевіряє сумісну роботу реальних компонентів (Repository + ORM + SQLite). На відміну від unit test, не використовує моки для бази. Окрема тестова БД унеможливлює пошкодження бойових даних.

46. Що перевіряє тест rollback? Перевіряє, що у разі помилки стан бази даних залишається повністю незмінним без проміжних спотворень.

47. Для чого використовуються fixtures? Для підготовки ізольованого тестового середовища (створення in-memory бази) та його коректного закриття.

48–50. Що роблять JOIN, GROUP BY та агрегатні функції? `JOIN` об'єднує таблиці за ключами; `GROUP BY` групує записи для розрахунків; `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` обчислюють статистичні агрегати безпосередньо на рівні СУБД.

20. Висновок

У лабораторній роботі №7 на базі предметної області «Каталог автомобілів» (варіант №5) успішно реалізовано надійний шар збереження даних (persistence layer). Розроблено реляційну схему SQLite з таблицями `manufacturers` та `cars` із відношенням one-to-many, первинними та зовнішніми ключами (`ON DELETE CASCADE`), а також обмеженнями `CHECK` і `UNIQUE`.

Реалізовано низькорівневу взаємодію з базою через стандарт Python DB-API з обов'язковою параметризацією запитів для запобігання SQL-ін'єкціям. Впроваджено сучасні декларативні моделі SQLAlchemy 2.0 (`Mapped`, `mapped_column`, `relationship`) та архітектурний патерн Repository, що ізолював доступ до бази від доменної логіки сервісу `CarCatalogDbService`. Для масової модифікації цін створено транзакційну операцію з гарантованим відкатом (`rollback()`), що на практиці продемонструвало дотримання вимог ACID (зокрема атомарності).

Структуру бази даних поставлено під контроль версій за допомогою інструменту міграцій Alembic (успішно додано стовпець `vin` з обмеженням `UNIQUE`). Працездатність усіх компонентів перевірено автоматизованими інтеграційними тестами у `pytest` з ізольованою in-memory базою даних (8 тестів виконано успішно, `100% passed`), а вихідний код успішно пройшов строгу типізацію `mypy`.